

Exqutor: Extended Query Optimizer for Vector-Augmented Analytical Queries

논문 제목: Exqutor: Extended Query Optimizer for Vector-Augmented Analytical Queries

저자: Hyunjoon Kim, Chaerim Lim, Hyeonjun An (연세대학교 BDAI Lab), Rathijit Sen (Microsoft Gray Systems Lab), Kwanghyun Park (연세대학교)

출처: arXiv:2512.09695v2 [cs.DB]

발행 연월: 2025년 12월 11일

요약

연구 문제

최근 대규모 언어 모델(LLM)의 도입과 확산에 따라 RAG(Retrieval-Augmented Generation) 파이프라인이 전자상거래, 의료, 추천 시스템 등 다양한 산업에서 널리 사용되고 있다. RAG는 LLM이 질문에 답하기 전에 외부 지식(문서, 이미지 등)을 검색해서 참고함으로써 "환각(hallucination)" 문제를 크게 줄일 수 있다. 이 과정에서 벡터 유사도 검색(VSS)은 핵심 기술이다. 그러나 현실의 분석 작업은 단순한 "top-k 유사 문서 찾기"를 넘어선다. 실제로는 벡터 검색 결과에 대해 여러 테이블 간의 조인, 날짜 필터링, 집계 등 복잡한 관계형 연산이 필요하다. 예를 들어, "주어진 이미지와 시각적으로 유사한 부품을 공급하는 업체를 찾고, 관련 고객 댓글과 할인율을 분석해 최적의 할인율을 제안하기" 같은 쿼리가 그것이다. 이런 쿼리를 본 논문은 **VAQ(Vector-Augmented Analytical Query)**라고 부른다.

핵심 기술 과제

범용 벡터 데이터베이스(pgvector, VBASE, DuckDB 등)는 VAQ를 지원하지만, 기존 쿼리 옵티마이저들은 벡터 유사도 검색의 **카디널리티(결과 행 수)를 정확하게 추정하지 못한다**. 현황은 다음과 같다:

- pgvector: 항상 전체 행의 33.3%로 추정
- VBASE: 항상 전체 행의 50%로 추정
- DuckDB: 항상 전체 행의 100%로 추정 (즉, 필터 효과 무시)

이러한 부정확한 추정으로 인해 옵티마이저는 잘못된 실행 계획(예: 불필요한 Full Table Scan, 비효율적 조인 방식)을 선택하게 되고, 결과적으로 쿼리 실행 시간이 수십 배 이상 길어진다.

본 논문의 접근법

본 논문은 **Exqutor**라는 확장 쿼리 옵티마이저를 제안한다. Exqutor는 기존 데이터베이스 시스템의 옵티마이저에 플러그인처럼 통합되어, 벡터 검색의 카디널리티를 정확하게 추정함으로써 더 효율적인 실행 계획을 세울 수 있게 한다.

본 논문은 두 가지 전략을 제시한다:

1. **ECQO(Exact Cardinality Query Optimization)**: 벡터 인덱스(HNSW)가 존재할 때, 쿼리 계획 수립 단계에서 실제로 인덱스 검색을 수행해 정확한 카디널리티를 얻는다. 계획 단계의 오버헤드는 1~2ms 수준으로 무시할 수 있으며, 검색 결과를 캐시하여 실행 단계에서 재사용한다.
2. **샘플링 기반 추정**: 벡터 인덱스가 없을 때, 데이터의 작은 표본을 추출해 유사도 조건을 만족하는 비율을 계산한 후, 이를 전체 데이터에 외삽(extrapolate)한다. 본 논문은 모멘텀 기반 적응적 샘플 크기 조정 알고리즘을 제안하여, 각 데이터셋의 특성에 맞게 자동으로 샘플링 전략을 최적화한다.

주요 기여 및 성과

1. **VAQ 벤치마크**: TPC-H와 TPC-DS를 벡터 데이터로 확장한, 최초의 벡터+구조화 데이터 통합 벤치마크를 제공한다.
2. **성능 개선**: 실험 결과에 따르면:
 3. pgvector에서 최대 약 1,000배 속도 향상
 4. VBASE에서 최대 약 10,000배 속도 향상
 5. DuckDB에서 1.5배~37.2배 속도 향상
 6. 벡터 인덱스가 없을 때도 적응적 샘플링으로 1.2~3.2배 속도 향상

7. **범용성**: pgvector(PostgreSQL 기반), VBASE(PostgreSQL 기반), DuckDB(임베디드 분석 DB) 등 서로 다른 아키텍처를 가진 세 가지 시스템에 성공적으로 통합되었다.
8. **오버헤드 최소화**: ECQO의 오버헤드는 1.88~1.96ms로 상대 오버헤드가 0.004% 수준으로 무시할 수 있다.
9. **확장성**: 데이터 크기가 증가해도 HNSW 인덱스의 로그 복잡도 덕분에 성능 향상이 일관되게 유지된다.

상세분석

1. 문제 정의 및 배경

1.1 RAG 파이프라인과 벡터 유사도 검색의 필요성

대규모 언어 모델(LLM)이 전자상거래, 의료, 추천 시스템 등 다양한 실제 응용에 통합되면서, **RAG(Retrieval-Augmented Generation)** 워크플로우가 업계 표준이 되었다. RAG의 핵심 아이디어는 다음과 같다:

- LLM이 사용자 질문에 답변을 생성할 때, 먼저 **외부 지식베이스에서 관련 정보(문서, 이미지, 데이터 등)**를 검색한다.
- 검색된 정보를 LLM의 맥락(context)에 포함시켜 답변을 생성한다.
- 이렇게 하면 LLM이 학습 데이터에 없는 새로운 정보나 최신 정보에 대해 답변할 수 있으며, "환각(hallucination)" 문제도 크게 줄어든다.

이 RAG 파이프라인에서 가장 핵심적인 연산이 **벡터 유사도 검색(Vector Similarity Search, VSS)**이다. 구체적으로:

1. 사용자의 질문이나 쿼리를 **수치 벡터(numerical vector)**로 변환한다. (예: 768차원의 BERT 임베딩)
2. 데이터베이스에 미리 저장된 **문서/이미지 벡터들** 중에서 쿼리 벡터와 가장 유사(거리가 가장 가까운)한 것들을 찾는다.
3. 검색 결과를 LLM에 전달하여 답변 생성에 활용한다.

1.2 벡터 데이터베이스의 두 가지 유형

전문(특화) 벡터 데이터베이스

Milvus, Pinecone, Qdrant, Weaviate, ChromaDB 등이 여기에 해당한다. 이들은 벡터 검색에 최적화되어 있고 매우 빠른 성능을 자랑한다. 하지만 근본적인 한계가 있다:

- 복잡한 SQL 쿼리(조인, 집계, 서브쿼리 등)를 지원하지 어렵다.
- 일반적으로 단일 컬렉션(테이블) 위에서만 검색한다.
- 메타데이터 필터링 정도만 지원할 수 있다.

범용 벡터 데이터베이스

pgvector(PostgreSQL 기반), VBASE(PostgreSQL 기반), AnalyticDB, DuckDB-VSS(DuckDB 기반) 등이 여기에 속한다. 이들은:

- 기존의 성숙한 관계형 데이터베이스 시스템에 벡터 검색 기능을 추가한 형태이다.
- 복잡한 SQL 쿼리, 다중 조인, 복잡한 필터링, 집계 연산을 모두 지원한다.
- 따라서 **VAQ(Vector-Augmented Analytical Query) 실행에 더 적합하다.**

본 논문은 이 범용 벡터 데이터베이스를 대상으로 한다.

1.3 최근접 이웃 검색과 벡터 인덱싱

벡터 유사도 검색의 기본 문제는 **최근접 이웃 검색(Nearest Neighbor Search)**이다:

KNN(K-Nearest Neighbor)

- 모든 벡터와 하나씩 거리를 계산해서 가장 가까운 k개를 정렬하여 반환한다.
- 결과가 정확하지만 데이터가 크면 매우 느리다. ($O(n \cdot d)$ 시간복잡도, n 은 벡터 수, d 는 차원)
- 실제 프로덕션 환경에서는 사용하기 어렵다.

ANN(Approximate Nearest Neighbor)

- 미리 구축한 인덱스를 활용해 근사적으로 빠르게 검색한다.
- 정확도를 약간 희생하지만 속도는 훨씬 빠르다. (보통 $O(\log n)$ 이상 $O(n)$ 이하)
- 실제 벡터 데이터베이스에서 주로 사용된다.

대표적인 ANN 인덱스 기법으로는 **HNSW(Hierarchical Navigable Small World)**가 있다:

- 계층적 그래프 구조를 만들어서, 검색 시 그래프를 탐색하면서 빠르게 유사한 벡터를 찾는다.
- top-k 검색과 범위 검색(range search)을 모두 지원한다.
- 삽입, 삭제 비용이 $O(\log n)$ 으로 효율적이다.
- 본 논문의 실험에서도 HNSW를 주요 인덱스로 사용한다.

1.4 카디널리티 추정의 중요성

데이터베이스 시스템은 쿼리를 **계획(planning)** 단계와 **실행(execution)** 단계로 나눈다:

계획 단계에서 쿼리 옵티마이저는:

1. 쿼리의 각 부분(스캔, 필터, 조인 등)을 실행하는 데 드는 비용을 추정한다.
2. 최소 비용이 되도록 실행 순서와 방식을 정한다.
3. 예: "어느 테이블을 먼저 스캔할지", "조인을 어떤 방식(Hash Join vs Nested Loop)으로 할지" 등.

이 비용 추정의 가장 핵심은 **카디널리티 추정(Cardinality Estimation)**이다. 즉, "어떤 연산의 결과로 몇 개의 행이 나올 것인가?"를 예측하는 것이다.

예시: 8000만 건의 데이터가 있는 부품(part) 테이블에서:

- "주어진 이미지와 유사한 부품 찾기" 조건을 적용하면 200건이 나온다고 하자.
- 옵티마이저가 "이 조건으로 200건이 나온다"를 알면, 작은 결과를 위해 **Nested Loop Join**을 선택할 것이다.
- 반대로 옵티마이저가 "이 조건으로 8000만 건 전부 나올 것"이라고 잘못 추정하면, 대용량 데이터를 위해 **Hash Join**을 선택하게 되고, 불필요한 메모리와 시간이 소비된다.

1.5 벡터 검색의 카디널리티 추정 문제

현존하는 범용 벡터 데이터베이스들은 모두 **벡터 유사도 검색의 카디널리티를 정확하게 추정하지 못한다**. 논문의 Table I에 따르면:

시스템	추정 방식	문제점
pgvector	항상 전체 행의 33.3% 고정 추정	실제 데이터나 임계값과 무관하게 같은 비율
VBASE	항상 전체 행의 50.0% 고정 추정	마찬가지
DuckDB	항상 전체 행의 100% 추정	벡터 필터의 효과를 전혀 고려하지 않음

예를 들어, 8000만 건의 데이터에서 벡터 유사도 조건으로 200건만 나와야 하는데:

- pgvector는 "약 2,667만 건이 나올 것"이라고 추정 ($2667/200 \approx 13$ 배 오차)
- VBASE는 "약 4,000만 건이 나올 것"이라고 추정 (20배 오차)
- DuckDB는 "8,000만 건 전부 나올 것"이라고 추정 (40만 배 오차!)

이렇게 실제와 동떨어진 추정을 하면 옵티마이저가 **완전히 잘못된 실행 계획을 세운다**. 그 결과 실행 시간이 수십 배에서 수천 배까지 늘어날 수 있다.

2. Exqutor의 핵심 기술: ECQO와 적응적 샘플링

2.1 ECQO(Exact Cardinality Query Optimization)

ECQO의 기본 아이디어는 간단하지만 강력하다: 쿼리 계획 수립 단계에서 실제로 벡터 검색을 일부 실행해서 정확한 카디널리티를 얻는다.

동작 원리:

1. VAQ가 들어온다. 예를 들어:

```
sql SELECT ps.ps_partkey, COUNT(*) FROM partsupp ps WHERE ps.ps_embedding <-> $1 < 0.5 -- 벡터 유사도 조건 GROUP BY ps.ps_partkey
```

2. 옵티마이저가 이 쿼리를 파싱하고 분석한다. 이때 벡터 범위 조건 `ps.ps_embedding <-> $1 < 0.5` 를 감지한다.

3. Exqutor의 ECQO 모듈이 활성화된다:

4. 벡터 인덱스(HNSW)가 존재하는지 확인한다.
5. 존재한다면, 쿼리의 실제 벡터(\$1)와 거리 임계값(0.5)을 사용해 HNSW 인덱스에 범위 검색을 수행한다.
6. 검색 결과로 나온 **정확한 후보 벡터 수**(예: 234개)를 카디널리티로 사용한다.
7. 이 정확한 카디널리티 234를 옵티마이저의 비용 모델에 반영한다.
8. 조인 순서 결정
9. 조인 방식 선택 (Nested Loop vs Hash Join)
10. 스캔 전략 선택 (Index Scan vs Sequential Scan)
11. 최적화된 실행 계획이 생성되고 **캐시에 저장**된다.
12. 실제 실행 단계에서 HNSW 검색을 다시 수행하지 않고, 캐시된 결과를 재사용한다.

왜 오버헤드가 작은가?

- HNSW 같은 계층적 그래프 인덱스는 로그 시간 복잡도로 동작하므로, 계획 수립 단계에서 호출해도 매우 빠르다. (보통 1~2ms)
- HNSW는 **결정론적(deterministic)** 알고리즘이므로, 같은 쿼리 벡터, 같은 인덱스 구조, 같은 파라미터(`ef_search` 등)에 대해 항상 같은 결과를 반환한다.
- 따라서 계획 단계의 검색 결과를 캐시하고 실행 단계에서 안전하게 재사용할 수 있다.

구현 방식:

- **pgvector, VBASE**: 플래너(planner)의 훅(hook)을 확장하여, 벡터 범위 조건을 인식하고 인덱스 검색을 트리거한다.
- **DuckDB**: 논리적 옵티마이저(logical optimizer)의 규칙을 수정하여 카디널리티 추정을 통합한다. DuckDB의 인프로세스(in-process) 아키텍처 덕분에 이 추정값이 하위 옵티마이저 단계로 효율적으로 전파된다.

2.2 벡터 인덱스가 없을 때의 샘플링 기반 추정

벡터 인덱스가 없으면 ECQO를 사용할 수 없다. (전체 데이터를 순차 스캔해야 하므로 비용이 크다) 이 경우 Exqutor는 **샘플링 기반 카디널리티 추정**을 사용한다.

기본 아이디어:

1. 전체 데이터의 작은 표본(샘플)을 추출한다.
2. 샘플에서 벡터 유사도 조건을 만족하는 행의 비율을 계산한다.
3. 이 비율을 전체 데이터에 적용해 카디널리티를 추정한다.

초기 샘플 크기 결정 (수식):

$$N = \text{ceil}((z^2 \times P_{\text{hat}} \times (1 - P_{\text{hat}})) / e^2)$$

여기서:

- **z**: 신뢰도 임계값 (95% 신뢰도 → $z = 1.96$)
- **P_hat**: 유사도 조건을 만족하는 데이터의 예상 비율 (기본값 0.5, 즉 50%)
- **e**: 허용 오차 (기본값 0.05, 즉 5%)

계산 예시:

$$\begin{aligned} N &= \text{ceil}((1.96^2 \times 0.5 \times (1 - 0.5)) / 0.05^2) \\ &= \text{ceil}((3.8416 \times 0.25) / 0.0025) \\ &= \text{ceil}(0.9604 / 0.0025) \\ &= \text{ceil}(384.16) \\ &= 385 \end{aligned}$$

즉, 약 385개 행의 샘플만으로도 95% 신뢰도에서 $\pm 5\%$ 오차 범위의 추정을 할 수 있다는 의미이다.

적응적 샘플 크기 조정 (Adaptive Sampling):

고정 샘플 크기는 모든 데이터셋에 최적이지 아닐 수 있다:

- 고차원 데이터: 벡터들이 고르게 분포하지 않아서, 고정 샘플이 대표성을 갖기 어렵다.
- 편향된 분포: 클러스터가 존재하는 데이터에서는 고정 샘플이 충분하지 않을 수 있다.

이를 해결하기 위해 Exqutor는 **모멘텀 기반 적응적 조정 알고리즘**을 제안한다:

Q-error 지표:

$$Q\text{-error} = \max(\text{Card_esti} / \text{Card_true}, \text{Card_true} / \text{Card_esti})$$

Q-error는 1에 가까울수록 정확하고, 1을 넘어갈수록 오차가 크다는 의미이다.

조정 규칙:

1. 최근 쿼리들의 Q-error와 현재 샘플링 비율을 사용해 조정 인자(δ)를 계산한다.
2. **모멘텀**을 적용해서 급격한 변동을 방지한다 ($m = 0.9$):

$$\delta_{\text{new}} = (1 - m) \times \delta_{\text{calc}} + m \times \delta_{\text{prev}}$$
3. **학습률**(η)을 감쇠시켜 점차 수렴을 가속화한다.
4. 50개 쿼리마다 샘플 크기를 업데이트한다.

실제 결과:

- DEEP, SimSearchNet++ 데이터셋: 시간이 지나면서 샘플 크기가 감소 (안정적인 분포이므로 작은 샘플로도 충분)
- SIFT 데이터셋: 샘플 크기가 증가 (더 복잡한 분포를 가져서 더 큰 샘플 필요)

3. VAQ 벤치마크: 벡터와 관계형 데이터의 통합**3.1 벤치마크의 필요성**

기존 ANN 벤치마크(예: ANN Benchmarks)는 다음 문제를 가진다:

- **단순성**: 단일 테이블에서 top-k 검색 성능만 평가한다.
- **현실성 부족**: VAQ는 여러 테이블 간 조인, 필터링, 집계를 포함하는데, 이를 평가하지 못한다.

본 논문은 **최초의 VAQ 벤치마크**를 제안한다. TPC-H와 TPC-DS라는 성숙한 표준 벤치마크를 확장하여 벡터 데이터를 추가한 것이다.

3.2 TPC-H 확장

TPC-H는 전자상거래 데이터베이스 벤치마크로, 8개 테이블과 22개의 복잡한 분석 쿼리를 포함한다.

본 논문은 다음과 같이 확장한다:

테이블 수정:

- **partsupp** 테이블에 두 개의 임베딩 컬럼 추가:
 - `ps_image_embedding` : 이미지 임베딩 벡터
 - `ps_text_embedding` : 텍스트 임베딩 벡터
 - `ps_tag` : 태그 정보 (멀티모달 특성 시뮬레이션)
 - **part** 테이블에 `p_text_embedding` 컬럼 추가

사용된 벡터 데이터셋:

데이터셋	차원	특징
DEEP	96	딥러닝 기반 이미지 특징 벡터
SIFT	128	이미지의 지역 특징 벡터 (Scale-Invariant Feature Transform)
SimSearchNet++	256	유사 이미지 탐지용 벡터
YFCC	192	Yahoo Flickr Creative Commons 멀티미디어 연구 데이터셋
WIKI	768	위키피디아 텍스트 임베딩

선택된 쿼리: TPC-H 22개 중 Q3, Q5, Q8, Q9, Q10, Q11, Q12, Q20을 선택하여 벡터 유사도 조건을 추가했다.

3.3 범위 검색(Range Search) vs Top-k 검색

핵심 특징:

본 벤치마크는 **top-k 검색이 아닌 범위 검색(range search)**을 사용한다:

```
ps_embedding <-> ${image_embedding} < ${D}
```

여기서:

- `<->` : 유클리드 거리 연산자
- `D` : 사용자가 정한 거리 임계값

왜 범위 검색인가?

- **Top-k 검색**: 항상 정해진 k개를 반환한다. 따라서 카디널리티가 항상 k로 고정되어, 옵티마이저의 도전이 적다.
- **범위 검색**: 임계값 D에 따라 반환되는 행 수가 극적으로 변한다. 임계값 0.3일 때는 200건, 0.5일 때는 50,000건 등. 따라서 카디널리티 추정이 훨씬 어렵고, 옵티마이저에 더 큰 도전이 된다.

범위 검색은 실제 현업에서도 더 일반적인 사용 패턴이다. (예: "0.5보다 유사한 이미지", "거리 50km 이내의 점포" 등)

3.4 TPC-DS 확장

TPC-DS는 TPC-H보다 더 복잡한 소매 데이터 분석 벤치마크이다:

- item 테이블에 `i_embedding` 컬럼 추가
- Q7, Q12, Q19, Q20, Q42, Q72, Q98 등 복잡한 쿼리에 벡터 조건 추가
- TPC-DS는 99개의 복잡한 쿼리를 가지므로, VAQ의 다양한 시나리오를 평가할 수 있다.

4. 실험 결과 및 성능 분석

4.1 실험 환경

- 하드웨어: Intel Xeon Gold 6530 (128 vCPU), 1.0 TB RAM
- 데이터베이스: pgvector, VBASE, DuckDB (with DuckDB-VSS)
- 벡터 데이터셋: DEEP, SIFT, SimSearchNet++, YFCC, WIKI
- 인덱스: HNSW (M=16, ef_construction=200, ef_search=400)
- 데이터 규모: TPC-H SF100 (약 80억 행), TPC-DS SF10 (약 11억 행)

4.2 ECQO 성능: 벡터 인덱스가 있는 경우

결과 요약 (Figure 4):

pgvector, VBASE, DuckDB 모두에서 Exqutor 적용 후 dramatic improvement:

시스템	최대 속도 향상	평균 향상
pgvector	~1,000배 (3 orders of magnitude)	약 100~500배
VBASE	~10,000배 (4 orders of magnitude)	약 500~3000배
DuckDB	37.2배	약 2~10배

실제 사례: pgvector Q3 쿼리 with DEEP 데이터셋

Exqutor 적용 전:

Full Table Scan (8000만 행)

→ Hash Join 3번

→ 총 실행 시간: 52,535초 (약 14.6시간!)

Exqutor 적용 후:

HNSW Index Scan (234행 반환)

→ Nested Loop Join (효율적)

→ 총 실행 시간: 15.571ms (0.016초)

속도 향상: $52,535 / 0.015571 \approx 3,368,000$ 배

왜 이렇게 큰 개선이 가능한가?

1. 정확한 카디널리티: 옵티마이저가 "벡터 필터로 234행만 나온다"는 것을 알게 됨.
2. Nested Loop Join 선택: 작은 결과셋에 적합한 조인 방식이 자동으로 선택됨.
3. 인덱스 우선: 벡터 필터가 먼저 적용되어 초기에 데이터를 대폭 줄임.
4. Index Scan 선택: Sequential Scan 대신 Index Scan으로 처리.

ECQO 오버헤드 분석 (Table II):

데이터셋	ECQO 오버헤드	실행 시간 단축	상대 오버헤드
DEEP	1.88ms	43.34초	0.0043%
SIFT	1.89ms	41.65초	0.0045%
SimSearchNet++	1.96ms	43.66초	0.0045%

오버헤드는 무시할 수 있는 수준이다.

4.3 샘플링 기반 추정: 벡터 인덱스가 없는 경우**결과 요약 (Figure 5):**

- 고정 샘플링: pgvector 기본 대비 1.2배~3.2배 속도 향상
- 적응적 샘플링: 고정 샘플링 대비 추가로 최대 1.4배 향상

적응적 샘플링의 이점:

고정 샘플은 모든 데이터셋에 최적이지 않다:

- DEEP, SimSearchNet++: 상대적으로 균일한 분포 → 적응적 샘플이 크기 감소 (더 효율적)
- SIFT: 클러스터링된 분포 → 적응적 샘플이 크기 증가 (정확도 향상)

샘플링 오버헤드:

고정/적응적 샘플링의 오버헤드는 28~73ms로, ECQO의 1~2ms보다는 크지만, 여전히 절약되는 실행 시간(26~47초)에 비해 무시할 수 있다.

4.4 다양한 워크로드에서의 성능**태그 기반 필터 + 벡터 검색 (Figure 7):**

YFCC 데이터셋에서 태그 필터(`ps_tag = 'nature'` 같은 조건)와 벡터 유사도 조건을 결합한 복합 쿼리:

- 최대 **301.5배 속도 향상**

멀티 벡터 쿼리 (Figure 8, 9):

서로 다른 벡터를 가진 두 테이블을 조인하는 복잡한 시나리오:

- partsupp 테이블: DEEP 임베딩 (96차원)
- part 테이블: WIKI 임베딩 (768차원)
- 결과: **1.07배~479.4배 속도 향상**

TPC-DS 기반 평가 (Figure 10):

TPC-H보다 훨씬 복잡한 분석 쿼리:

- 최대 **109.6배 속도 향상**
- 더 복잡한 조인과 집계를 포함하는데도 일관된 개선 보임

4.5 기존 방법과의 비교**SelNet과의 비교 (Figure 12):**

SelNet은 신경망을 사용한 학습 기반 선택도 추정기이다.

항목	Exqutor	SelNet
속도	기준	Exqutor가 최대 16.1배 더 빠름
학습 필요	No	Yes (오프라인 학습 필수)
확장성	우수 (인덱스 크기 선행)	낮음 (데이터 커질수록 학습 비용 증가)
복잡성	낮음 (즉시 사용)	높음 (모델 관리 필요)
범용성	우수 (인덱스만 있으면 작동)	특정 데이터셋에 튜닝 필요

4.6 확장성 평가 (Figure 14)

Scale Factor를 1, 10, 100으로 변경하며 데이터 규모 테스트:

pgvector 기본:

- 데이터 크기에 비례해 거의 선형적으로 실행 시간 증가
- SF 1에서 SF 100으로 100배 증가 → 실행 시간도 약 100배 증가

Exqutor 적용:

- HNSW 검색은 데이터 크기에 대해 로그 복잡도를 가지므로, 데이터가 증가해도 성능 향상이 일관되게 유지됨
- SF 100에서도 최대 1.31배 속도 향상을 유지
- 상대적으로 더 큰 데이터셋에서 더 큰 이득

5. 한계점 및 향후 과제

5.1 고차원 공간에서의 도전

고차원(특히 768차원 WIKI) 데이터에서:

- 샘플링 오버헤드가 거리 계산 비용 증가로 인해 커질 수 있다.
- 벡터 간 거리 분포가 균일하지 않아(curse of dimensionality), 샘플링 기반 추정의 정확도가 떨어질 수 있다.

5.2 비용 모델의 한계

기존 데이터베이스의 비용 모델:

- ANN 인덱스(HNSW)의 성능 특성을 제대로 반영하지 못함.
- 정확한 카디널리티가 있어도, 비용 추정이 부정확하면 여전히 최적 계획을 선택하지 못할 수 있다.

5.3 향후 연구 방향

1. **고차원 효율적 샘플링**: 고차원 공간에서의 대표성 있는 샘플 추출 방법 개발.
2. **정제된 비용 모델**: ANN 인덱스의 특성을 반영한 더 정확한 비용 추정 함수.
3. **동적 샘플 조정**: 쿼리 워크로드에 따라 샘플링 전략을 동적으로 조정.

6. 관련 연구 (Related Work)

6.1 필터링된 벡터 검색

전문 벡터 DB 접근법:

- Milvus, Qdrant: 메타데이터 필터링 지원
- ACORN, SeRF, HQANN, diskANN: ANN 인덱스에 속성 필터링 통합

한계: 모두 단일 테이블 내 검색에만 적용 가능. 복잡한 멀티 테이블 조인에는 효과 없음.

6.2 범용 벡터 DB의 쿼리 최적화

기존 시스템들:

- AnalyticDB: 비용 기반 모델 사용
- SingleStore: 인덱스 기반 필터 통합

한계: 단순 필터 쿼리에 집중. 복잡한 멀티 조인, 중첩 쿼리에는 비효율적.

6.3 샘플링 기반 선택도 추정

기존 연구:

- 랜덤 샘플링을 사용한 조인 크기 추정
- 적응적 샘플링 전략

본 논문의 기여: 벡터 유사도 검색에 특화된 샘플링 기반 추정은 Exqutor가 최초.

7. 결론

본 논문의 핵심 기여:

1. **ECQO:** 벡터 인덱스를 활용한 정확한 카디널리티 추정으로, 최대 4 orders of magnitude(10,000배)의 속도 향상 달성. 오버헤드는 1~2ms 수준으로 무시할 수 있음.
2. **적응적 샘플링:** 벡터 인덱스가 없는 환경에서도 모멘텀 기반 동적 샘플 크기 조정으로 1.2~3.2배 속도 향상. 각 데이터셋의 분포 특성에 자동으로 적응.
3. **VAQ 벤치마크:** TPC-H/TPC-DS를 확장한 최초의 벡터+구조화 데이터 통합 벤치마크 제공. 범위 검색 기반으로 현실적인 VAQ 워크로드 평가 가능.
4. **범용성 입증:** pgvector(PostgreSQL), VBASE(PostgreSQL 기반), DuckDB(임베디드 OLAP) 세 가지 서로 다른 아키텍처의 시스템에 성공적으로 통합. 각 시스템의 고유한 옵티마이저 구조에 맞게 구현.
5. **산업 영향:** 현업의 VAQ 성능 문제를 실질적으로 해결. LLM + 벡터 임베딩 + 관계형 분석의 시대에 필수적인 기술.

추가 제기 문제

1. **샘플링의 일반화 문제:** 현재 샘플링은 임의의 거리 임계값을 가정한다. 하지만 동일한 쿼리 벡터에 대해 여러 임계값이 연속으로 쿼리될 때, 각 임계값별로 새로운 샘플을 추출해야 하는가? 아니면 캐시된 샘플로부터 선택도를 외삽할 수 있는가?
2. **고차원에서의 카디널리티 추정:** WIKI(768차원) 같은 매우 고차원 데이터에서 샘플링 기반 추정의 정확도가 급격히 떨어진다. 이를 해결하기 위한 전문화된 샘플링 전략이 필요한가?
3. **동적 임계값 선택:** 사용자가 "유사한 부품을 찾는데 몇 개가 나올지 모르는" 상황에서, 임계값을 자동으로 조정해주는 기능이 필요할 수 있다. Exqutor가 이를 지원할 수 있는가?
4. **멀티 벡터 조인 최적화:** 여러 테이블에 각각 다른 벡터가 있고 모두 조인에 참여할 때, 조인 순서 결정이 복잡해진다. ECQO만으로는 충분한가, 아니면 추가 휴리스틱이 필요한가?
5. **벡터 업데이트의 영향:** 벡터 데이터가 주기적으로 업데이트될 때(예: 매일 새로운 이미지 임베딩 추가), HNSW 인덱스의 재구성 비용과 캐시된 카디널리티의 유효성을 어떻게 관리할 것인가?

6. **다중 임계값 쿼리**: 하나의 쿼리에 여러 개의 벡터 유사도 조건이 있을 때(예: 이미지 유사도 < 0.5 AND 텍스트 유사도 < 0.3), 조건 간의 상관관계를 어떻게 모델링할 것인가? 독립성을 가정하면 오차가 누적될 수 있다.